ChatGPT

Product Requirements Document for a Sports-Arbitrage Odds Scanner with Telegram Alerts

Executive summary

You're building a **Model A odds scanner**: it **monitors odds across multiple Iowa-legal sportsbooks**, detects **risk-screened arbitrage opportunities**, and **alerts you on Telegram** fast enough that you can place the bets manually before the market moves.

This PRD assumes: - You want **US/Iowa-legal operator coverage only** (no offshore books). - You want **alert-first workflows** (not auto-betting). - You want a system you can operate reliably with **clear KPIs, auditability, and controls**.

For Iowa compliance and "accounts you can open in Iowa," this PRD uses the Iowa regulator's list of **advance deposit sports wagering operators** (online sportsbooks/exchanges) as the allowed book set. [1] It also treats Iowa's published "Approved Sports Wagers" constraints as rules your product should support as **filters**, especially the prohibition on **player proposition wagers for individual athletes from Iowa colleges**. [2]

Goals, success metrics, and scope

Product goals

You will: - Detect **true arbs** across Iowa-legal operators with **low false positives**. - Deliver actionable alerts to Telegram with **clear staking guidance** and **risk flags**. - Stay stable under vendor rate limits and messaging limits. - Keep the system auditable so you can explain "why this alert fired" later.

Explicit non-goals

You will not: - Auto-place wagers (that is Model B / execution automation). - Circumvent sportsbook controls or scrape private endpoints (you'll use vendor feeds and official/public APIs where available). - Guarantee profits (you will reduce error sources; you can't control line movement and bet acceptance).

Success metrics (KPIs)

Define these as dashboard-visible metrics and weekly review items.

Arb detection quality - **Arb detection rate**: `# valid arbs alerted / # arbs detected by engine` (target ≥ 95% validity in MVP; validity verified by post-hoc odds snapshot consistency + manual confirmation sampling). - **False positive rate**: `% of alerts that are not actionable` due to stale lines, mismatched events/markets, suspended

markets, or stake constraints (target ≤ 5% in MVP). - **Duplicate alert rate**: `% of alerts that repeat the same opportunity without meaningful change` (target ≤ 10% in MVP via dedupe + throttling).

Speed - **Time-to-alert (TTA)**: p50/p95 from "odds update ingested" → "Telegram delivered"
- Prematch target: p50 ≤ 3s, p95 ≤ 8s (engine + alerting).
- Live target (if enabled): p50 ≤ 2s, p95 ≤ 5s (only realistic with streaming feeds; many polling feeds update slower as a vendor constraint). For example, some polling odds feeds publish on the order of ~40–60 seconds for common markets. [3]

Reliability - **Uptime** (scanner + alerting): ≥ 99.9% monthly for MVP. - **Vendor error budget**: track 4xx/5xx/429 rates per vendor and stay under documented rate limits (examples: The Odds API publishes 30 req/s; OpticOdds publishes per-15s windows; Telegram bots have broadcast/message limits). [4]

Operational usefulness - **Median "decision time"**: time between alert delivery and your manual "acknowledge/ignore" action (target: trending down as alert clarity improves). - **Post-alert conversion** (optional, manual): `% of alerts you actually bet` (trend metric; not a "success" KPI by itself because it depends on your strategy).

Scope boundaries

Sports and markets for MVP - 2-way markets: Moneyline, spread, total (full game).
- 3-way markets: Soccer-style 1X2 (home/draw/away) where available.

Iowa legality filters - Iowa's "Approved Sports Wagers" document states that **prop wagers for individual athletes from colleges in Iowa are strictly prohibited**. Your product must support filtering that prevents alerting on these markets when a team from an Iowa college participates. [5]

Target users and personas

Primary persona: You as the product owner and power user

You want: - Fast, trustworthy alerts. - Control over thresholds and risk rules. - A simple operational loop (scan → alert → decide → log outcome).

Pain points the product must solve: - Odds feeds differ in naming and timing. - "Good-looking arbs" vanish because lines move or markets suspend. - Alerts become spam if you don't de-duplicate and throttle.

Secondary persona: Analyst-mode user (still you, different mindset)

You want: - Historical logs to answer: *Which books move first? Where do false positives come from? What filters improve hit rate?* - Debug tooling: *Show me the exact odds objects used, timestamps, and matching keys.*

Optional future persona: Small group (friends/partners) on the same Telegram channel

You want: - Role-based access (admin vs reader). - Guardrails to prevent flooding Telegram and hitting bot limits. Telegram documents broadcast/message limits and 429 behavior. [6]

Functional requirements and user stories

Prioritized feature set

The table below describes **what you ship** and **how you validate it**. (P0 = MVP)

Priority	Feature	User story	Acceptance checks
P0	Supported operator set (Iowa-legal)	As the product owner, you can select which Iowa-licensed operators power scanning and alerts.	Operator list matches Iowa regulator list for advance deposit sports wagering operators. [1]
P0	Odds ingestion from a vendor feed	As a user, you can connect a sportsbook-odds API feed and pull odds for selected sports/markets.	Ingestion runs continuously within vendor rate limits and logs last successful pull/stream time per source. (Examples of published rate limits exist for The Odds API and OpticOdds.) [7]
P0	Normalization layer	As a user, you get comparable odds objects across books regardless of input quirks.	Canonical event/market schema; deterministic normalization; unit tests for conversions and naming.
P0	Event + market matching	As a user, you only compare “like with like” (same game, same market, same period).	Matching precision $\geq$ 99% on supported leagues; mismatches quarantined with explainable reasons.
P0	Arb detection engine for 2-way	As a user, you receive an alert when a 2-outcome market yields a positive arb after filters.	Engine computes implied probability sum < 1 (decimal form) and records arb% + books + timestamps.
P0	Arb detection engine for 3-way	As a user, you receive an alert when a 3-outcome market (1X2) yields a positive arb after filters.	Engine evaluates 3-way implied probability and produces stakes for all outcomes.

Priority	Feature	User story	Acceptance checks
P0	Stake sizing	As a user, you get stake suggestions based on your bankroll rules and per-book caps.	Produces non-negative stakes, respects min/max, rounds to your configured increments.
P0	Telegram alerts	As a user, you receive a clear Telegram message with the arb and staking plan.	Uses Telegram Bot API `sendMessage` correctly and handles errors, including 429 with backoff. [8]
P0	Dedupe & throttling	As a user, you don't get spammed by repeated updates.	Same arb key within TTL does not re-alert unless meaningful change. Throttling respects Telegram per-chat limits. [9]
P0	Risk checks & filters	As a user, you avoid risky/ invalid arbs (stale, suspended, illegal props, etc.).	Filters block restricted Iowa college player props. [5]
P1	Manual override controls	As a user, you can pause/ resume scanning or alerts from Telegram commands or dashboard.	Supports updates via webhook or long polling; webhooks require SSL and supported ports as documented. [9]
P1	In-app dashboard	As a user, you can configure sources, thresholds, and view alert history.	Auth-guarded UI, shows KPIs, logs, and system health.
P1	Account/bankroll management	As a user, you can track balances/limits per operator and the system uses them in stake sizing.	Manual entry + CSV import; produces warnings when balances are insufficient.
P2	Multi-vendor fallback	As a user, you can switch feeds if a vendor is down or delayed.	Hot-switch supported with feature flags; "vendor health score" drives failover.
P2	Deep links (optional)	As a user, you can jump from alert to a sportsbook betslip (where available).	Links are per-operator and disabled by default (ToS and UX variability). Some data vendors advertise deep linking features. [10]

Iowa-legal operator set (source-of-truth)

Your allowed "books" set for Iowa should come from the regulator's licensed operator list. The Iowa regulator explains that it licenses sports wagering operators associated with Iowa casinos and lists the specific advance deposit operator licensees. [1]

Use the following as the MVP operator set (and configure per-user enable/disable toggles):

- Caesars Sportsbook [11] [1]
- Bally Bet [12] [1]
- FanDuel [13] [1]
- BetMGM [14] [1]
- Circa Sports [15] [1]
- DraftKings [16] [1]
- Q Sportsbook [17] [1]
- Fanatics Sportsbook [18] [1]
- bet365 [19] [1]
- theScore Bet [20] [1]
- BetRivers [21] [1]
- Hard Rock Sportsbook [22] [1]
- Sporttrade [23] [24]

Data sources, normalization, and Telegram alerting design

Data sources strategy

You have two practical options for Model A:

Option A: Odds aggregation vendors (recommended for MVP)
Most Iowa sportsbooks do not offer widely-available public odds APIs for third-party developers. For MVP speed, you will typically integrate an odds vendor that already aggregates and normalizes sportsbook odds.

Option B: Enterprise/official feed providers (best "primary source," hardest procurement)
Enterprise providers (often serving sportsbooks directly) push official/low-latency feeds via message buses and proprietary integrations (examples include real-time pushed feeds over AMQP and XML). [25]
These can be excellent as "reference lines," but they may not represent the *exact* odds at each Iowa sportsbook—so they complement, not replace, cross-book scanning.

Odds API vendors comparison (3–5)

This table focuses on what matters for your product: **delivery model, freshness/latency constraints, rate limits, formats, and cost signals**. "Latency" often comes from either (a) vendor-published update intervals/delays, or (b) whether you get true streaming/push.

Vendor	Delivery model	Latency / freshness signal	Coverage signal	Formats	Rate limits	Cost signal
The Odds API [26]	REST polling. [27]	Published update intervals for featured markets: prematch ~60s; in-play ~40s. [3]	“US bookmakers” list includes major brands; API supports US region queries. [28]	American or decimal via `oddsFormat`. [27]	30 req/s published rate limit. [29]	Public tiers: free 500 credits/mo; paid tiers (example: 20K credits $30/mo). [30]
OpticOdds [31]	REST polling + SSE streaming + RabbitMQ queue ingestion. [32]	No WebSockets; provides SSE streaming for real-time updates and a queue-based option for scale. [33]	Vendor positions itself as broad sportsbook coverage; ingestion docs emphasize fixtures/markets endpoints and streaming/queue delivery. [32]	Marketing indicates JSON/XML. [34]	Published per-15s limits: 2500 req/15s for most endpoints; 250 new streaming connections/15s; 10 historical/15s. [35]	Pricing is “tailored”/sales-led on the public site (plan details not posted). [36]
SportsDataIO [37]	Aggregated odds feed (API-based), plus documented data model and line-movement endpoints. [38]	Positions as “live odds updates in real-time” with line movement timestamps (vendor-defined). [38]	Publishes a “Sportsbooks Covered” list including several major US brands. [39]	Publicly states JSON or XML support for its APIs. [40]	Vendor messaging emphasizes “unlimited API calls” (details likely contract-specific). [40]	Free trial exists; public pricing for the odds product is not posted on the odds page. [41]

Vendor	Delivery model	Latency / freshness signal	Coverage signal	Formats	Rate limits	Cost signal
TheRundown [42]	REST + optional WebSocket on real-time tiers. [43]	Documents tier-based "data delay" (e.g., 5 min on free; real-time on higher tiers) and burst limits. [44]	Docs claim real-time odds from 15+ sportsbooks. [45]	API-based JSON responses implied (docs + headers); supports WebSocket delivery when enabled. [44]	Docs publish tier defaults: burst limit and data-point metering; WebSocket access varies by tier. [44]	Marketing shows plan starting points (free and paid tiers). [46]
Unabated [47]	WebSocket push + snapshot endpoint combo. [48]	Describes "low-latency" WebSocket updates; reduces polling. [48]	Advertises real-time odds from 25+ sportsbooks. [49]	Not clearly advertised on the landing page; API docs describe snapshot + websocket patterns. [50]	Docs state "no call limits or API throttling" but recommend not polling more than 1/sec when nothing changes. [51]	Publishes starting price $3,000/ month for personal use. [49]

Vendor recommendation for MVP:
Start with **one** vendor that gives you (1) Iowa-legal operator coverage, (2) stable latency, and (3) clean IDs for events/markets. If you prioritize "quick build" and predictable constraints, a polling vendor with published update intervals can be easier to operate. [52]
If you prioritize live speed, look for **push/streaming** delivery (WebSocket/SSE/queue) so you stop wasting quotas and reduce time-to-alert. You can see multiple vendors emphasize streaming delivery (SSE, RabbitMQ, WebSocket) as their scale pattern. [53]

Required fields and normalization rules

Canonical schema (what every odds record becomes)

At minimum, the arb engine needs:

Event identity - `event_id` (vendor event ID, plus your internal canonical ID) - `sport`, `league` - `start_time_utc` - `home_team`, `away_team` (canonical) - `event_status` (scheduled/live/final/ suspended)

Market identity - `market_type` (moneyline/spread/total/1x2) - `period` (full game / 1H / 1Q / etc) - `point` (spread/total line; null for moneyline/1x2) - `selection` (home/away/draw/over/under)

Pricing - `odds_format` (internal decimal) - `price_decimal` - `price_american` (optional, for display) - `last_update_ts` (vendor timestamp) - `source_operator` (book ID/key) - `is_live` (boolean) - `is_suspended` (boolean, if vendor provides)

Constraints (for risk checks + stake sizing) - `min_stake`, `max_stake` (if available) - `liquidity` / `limit` fields (if available; some vendors explicitly provide limit/liquidity for certain books). [35]

Normalization rules (deterministic)

- **Time**: store all times in UTC; keep the raw vendor timestamp too. OpticOdds states it uses ISO 8601 timestamps (timezone may vary but ISO 8601 normalizes representation). [35]
- **Odds conversion**: convert everything to **decimal** for arb math; keep American for UI if you prefer.
- **Rounding**: defer rounding until display and stake sizing; store raw floats/decimals precisely.
- **Team/name canonicalization**: use a league-specific mapping table (e.g., "LA Rams" vs "Los Angeles Rams"). Avoid fuzzy matching as the primary method if the vendor supplies stable IDs.
- **Market mapping**: normalize vendor market IDs into your own controlled enum:
- 2-way: `MONEYLINE_2W`, `SPREAD`, `TOTAL`
- 3-way: `MONEYLINE_3W` / `1X2`
- **Main line vs alternates**: store `is_main_line` if the source provides it; otherwise compute based on "closest to consensus" heuristics and allow user override.

Arb detection logic (2- and 3-way)

Your system should compute arbs using decimal odds.

- **2-way arb condition**:
  `1/oddsA + 1/oddsB < 1`
- **3-way arb condition**:
  `1/oddsHome + 1/oddsDraw + 1/oddsAway < 1`

Then compute: - `arb_percent = (1 - implied_sum) * 100` - Stake sizing to equalize payout across outcomes, subject to bankroll and constraints.

Filters and risk checks (must-have)

These checks prevent "paper arbs" that waste your time:

Freshness and suspend checks - Require both sides (or all 3 outcomes) to have `last_update_ts` within a max age window (e.g., $\leq$ 15s for streaming feeds, $\leq$ 90s for polling feeds). - Block if any outcome is suspended/closed, or treated as stale.

Legality and Iowa restrictions - Filter out prohibited player props involving Iowa colleges. Iowa's published rule note explicitly restricts these wagers. [2]
- Provide a rule toggle: "Hide Iowa-college player props always."

Stake feasibility - Block if suggested stakes violate min/max on any book (when limits are provided) or exceed your configured per-book cap. - Block if required stake on any book exceeds your available “balance bucket.”

Dedupe - Key an arb as: `(canonical_event_id, market_type, period, point, outcome_book_map)` and dedupe within a TTL.

Telegram integration design

Integration options (table)

Telegram supports bots via HTTPS requests, and documents two primary ways to receive updates: long polling (`getUpdates`) and webhooks (`setWebhook`). [6]

Option	Best for	Latency	Operational complexity	Notes
Direct Bot API HTTPS calls (send-only)	MVP alerts where you don’t need inbound commands	Low	Low	Bot API requests use `https://api.telegram.org/bot<token>/METHOD_NAME` over HTTPS. [54]
Bot API + webhook receiver	Alerts + inbound commands (/pause, /resume, /status)	Low	Medium	Webhooks require a valid SSL cert; Telegram documents supported ports (443/80/88/8443). [9]
Bot API + long polling	Alerts + inbound commands without hosting a public webhook	Medium	Low–Medium	Simpler networking; can be less reliable if your process restarts often. Telegram documents long polling behavior and offsets. [9]
“Paid broadcast” mode	Very large subscriber broadcasting	Low	Medium	Telegram documents enabling higher broadcast rates for a fee and also documents default broadcast limits. Likely unnecessary for your scanner. [6]

Rate limits, throttling, retries, and delivery confirmation

Telegram documents practical broadcast limits and what triggers 429 errors: - In a single chat, avoid > ~1 msg/sec. - In groups, avoid > 20 msgs/min. - Bulk notifications: ~30 msgs/sec unless paid broadcasts are enabled. [9]

Throttling design - Implement a per-chat token bucket: 1 message/sec steady-state. - Add a global limiter (e.g., 20 msg/sec) even if you only message yourself—this prevents runaway loops.

Retries - On HTTP 429, read `Retry-After` and pause accordingly (Telegram and TheRundown both document this pattern for 429 cases). [55]
- On transient 5xx, exponential backoff with jitter (max retries: 3–5 per message).

Delivery confirmation - `sendMessage` returns the sent Message object on success. [54]
- Log the returned `message_id`, `chat_id`, timestamp, and the arb key so you can correlate later.

Message format requirements

Your alert must answer, in one glance: - What is the event and market? - What books and odds create the arb? - What is the arb% and expected profit? - What stakes should you place (rounded)? - What risk flags exist (stale odds, limits unknown, live market, etc.)?

Telegram's `sendMessage` endpoint supports text up to 4096 characters and formatting controls (`parse_mode`, entities). It also supports `link_preview_options` so you can disable link previews. [56]

Sample Telegram alert message

```
 ARB FOUND (2-way) — +1.42%

NBA • Lakers @ Warriors • Moneyline • Full Game
Start: 2026-03-09 19:10 CT

Best prices:
• Book A: Lakers +124
• Book B: Warriors +121

Stake plan (equal payout):
• Bet $214.50 on Lakers @ +124 (Book A)
• Bet $225.00 on Warriors @ +121 (Book B)

Locked profit (est.): $6.09
Risk checks:
 Both odds updated < 10s ago
⚠ Limits unknown (assume max $250 per bet)
 Live market: OFF (prematch only)

Actions: /ack 8f3c2  |  /mute 30m  |  /pause
```

Sample JSON payload (Telegram Bot API `sendMessage`)

```
{
  "chat_id": 123456789,
  "text":
"🚨 ARB FOUND (2-way) — +1.42%\n\nNBA • Lakers @ Warriors • Moneyline • Full
```

```
Game\nStart: 2026-03-09 19:10 CT\n\nBest prices:\n• Book A: Lakers +124\n• Book
B: Warriors +121\n\nStake plan (equal payout):\n• Bet $214.50 on Lakers @ +124
(Book A)\n• Bet $225.00 on Warriors @ +121 (Book B)\n\nLocked profit (est.):
$6.09\nRisk checks:\n Both odds updated < 10s ago\n⚠ Limits unknown (assume max
$250 per bet)\n\nActions: /ack 8f3c2  |  /mute 30m  |  /pause",
  "parse_mode": "HTML",
  "link_preview_options": {
    "is_disabled": true
  },
  "disable_notification": false
}
```

Telegram documents the HTTPS request pattern and the `sendMessage` parameters (including `parse_mode`, `link_preview_options`, and `disable_notification`). [56]

Architecture, performance, security, and operations

Reference architecture

You want clear separation between ingestion, normalization, detection, and alerting. This avoids the common failure mode where your alerting logic becomes entangled with vendor quirks.

```
flowchart LR
  subgraph Sources
    V1[Odds Vendor API\n(polling/streaming)]
    V2[Optional second vendor\n(failover)]
  end

  subgraph Ingestion
    C1[Collectors\npollers/stream clients]
    SCHED[Scheduler\ncron/worker loop]
  end

  subgraph Processing
    NORM[Normalizer\ncanonical schema]
    MATCH[Event & Market Matcher]
    ARB[Arb Engine\n2-way + 3-way\nstake sizing + risk checks]
  end

  subgraph Storage
    DB[(Postgres\nodds snapshots + alerts)]
    CACHE[(Redis\nhot cache + dedupe)]
  end

  subgraph Delivery
    ALERT[Alert Service\nTelegram sender\nthrottle + retry]
```

```
  UI[Dashboard\nconfig + history]
end

V1 --> C1
V2 --> C1
SCHED --> C1
C1 --> NORM --> MATCH --> ARB
NORM --> CACHE
MATCH --> CACHE
ARB --> DB
ARB --> ALERT
ALERT --> DB
DB --> UI
CACHE --> ARB
```

Recommended stack (pragmatic MVP)

Backend - Python (fast iteration for odds math + data pipelines) or TypeScript (if you prefer).
- FastAPI (API + dashboard backend) + background workers.

Storage - PostgreSQL (odds snapshots, alerts, configs, audit trails). - Redis (dedupe keys, hot odds cache, rate limiting tokens).

Queue / scheduling - A worker queue (Celery/RQ) or a lightweight job runner.
- If you use OpticOdds queue ingestion (RabbitMQ), you can use it directly as a transport for odds updates. OpticOdds documents a RabbitMQ ingestion method and connection settings. [57]

Hosting - Start with one small VM + managed Postgres, or container hosting (ECS/Kubernetes) when you need scale. - Put the collector close to US endpoints (us-east) to reduce latency variance.

Performance, latency, and scalability targets

Set targets your system controls, and treat vendor delay as an external constraint.

Internal system targets - Collector → Normalizer: p95 < 200ms per update batch. - Normalizer → Arb engine: p95 < 300ms for a “changed markets” batch. - Arb engine compute budget:
- MVP: up to ~50k market outcomes per cycle (prematch major sports) with p95 < 1s per cycle. - Telegram sender: p95 < 500ms (excluding Telegram network time), with strict throttle.

External constraints you must design around - Some polling feeds publish update intervals on the order of tens of seconds for common markets. [3]
- Some feeds meter usage by “data points” and apply tier-based delays and burst limits; you must monitor headers and throttle accordingly. [44]
- Telegram enforces chat/group/broadcast message limits and returns 429 errors if you exceed them. [9]

Security, compliance, and anti-abuse

Compliance guardrails - Only include Iowa-licensed operators in the default configuration (source: Iowa regulator list). [1]
- Implement market filters to respect published Iowa restrictions on certain college-related props. [5]

Credential management - Store vendor API keys and Telegram bot token in a secrets manager (not in the DB as plaintext). - Encrypt config-at-rest; restrict dashboard with MFA where possible.

Rate limits - Implement vendor-specific limiters using the published limits where provided: - The Odds API: 30 req/s. [29]
- OpticOdds: per-15s windows with different endpoint classes. [35]
- TheRundown: tier-based burst limits plus metered data points. [44]
- Implement Telegram throttling according to documented guidance to avoid 429. [9]

Anti-abuse / operational safety - Fail closed on uncertainty: if matching confidence drops or odds are stale, do not alert. - Add circuit breakers: if vendor errors spike, pause scanning and alert you once ("vendor degraded") instead of spamming.

Monitoring, logging, and observability

Golden signals per component - Collectors: request success rate, 429 rate, vendor latency, ingestion lag. - Normalizer: invalid record count, mapping misses (unknown teams/markets). - Matcher: match precision proxy (conflict rate, unmapped events). - Arb engine: compute time, arbs found, arbs filtered, false positive feedback labels. - Alerting: send success rate, throttle drops, retry count, Telegram 429 rate.

Structured logs - Every alert must log: - arb key - input odds objects (references to DB snapshot IDs) - filter decisions and reasons - Telegram message_id on success

TheRundown specifically documents usage headers like `X-Datapoints` and tier/limit headers that you should capture in logs if you use that vendor. [44]

Testing and QA plan

Unit tests (math correctness) - Odds conversion (American ↔ decimal). - 2-way and 3-way arb detection. - Stake sizing with rounding and min/max constraints. - Dedupe key stability.

Integration tests (vendor adapters) - Mock API responses for each vendor adapter. - Validate schema mapping is deterministic and idempotent. - Simulate 429 responses and verify backoff.

End-to-end tests - Run a staging environment that: - ingests limited sports/markets - triggers synthetic arbs from replayed snapshots - asserts Telegram payload formatting

Regression dataset - Store real odds snapshots in Postgres and build a replay harness so you can reproduce alert logic changes.

Roadmap, rollout, operational runbook, and acceptance criteria

Implementation roadmap (milestones, effort, roles)

Assume 1 product owner (you) plus 1–2 engineers. Estimate in “engineering weeks” (EW) for MVP.

Milestone	Scope	Est. effort	Roles
Foundation	Repo, CI, DB schema, config model, basic dashboard skeleton	1–2 EW	Backend engineer
Vendor adapter v1	Integrate one odds vendor (polling or streaming), store normalized odds snapshots	2–3 EW	Backend / data engineer
Normalization + matching	Canonical schema + event/market matching + mapping tables	2–3 EW	Backend / data engineer
Arb engine MVP	2-way + 3-way detection, stake sizing, filters, dedupe	2–3 EW	Backend engineer
Telegram alerting	Bot token, chat setup, message templates, throttling, retries	1–2 EW	Backend engineer
Observability	Metrics, dashboards, alerting, runbooks	1–2 EW	Backend + DevOps
Beta hardening	False positive reduction, tuning, usability, caching, load tests	2–4 EW	Backend engineer + you (QA)

Rollout plan

Phase 0: Local dry run - Use a single sport/league. - Log arbs but do not alert.

Phase 1: Private Telegram alerts - Enable alerts to one private chat. - Add `/ack` and `/mute` commands (optional) using long polling or a webhook. Telegram documents both approaches. [9]

Phase 2: Tighten filters - Add stale detection thresholds and Iowa rule filters. - Add per-book bankroll caps and per-market exclusions.

Phase 3: Add second vendor (optional) - If you need lower delay or coverage, add a fallback vendor and implement failover on “vendor health score.”

Operational runbook (what you do when things break)

Incident: vendor outage / high error rate 1. Collector detects error spike or ingest lag. 2. Circuit breaker pauses that vendor. 3. Telegram sends one “degraded vendor” notice (throttled). 4. Dashboard shows vendor status and ETA to auto-retry.

OpticOdds explicitly provides a status page reference for outages, which is helpful operationally if you use that vendor. [35]

Incident: rate limit hits - Telegram: comply with 429 and reduce send rate; keep a queue. Telegram documents limits and that exceeding them yields 429 errors. [9]
- Odds vendor: apply backoff and reduce polling frequency; cache lower-volatility endpoints. The Odds API explicitly recommends combining markets and caching endpoints that don't change often to reduce 429s. [29]
- TheRundown: watch `X-Datapoints-*` and burst limits; switch to delta endpoints or WebSocket tiers when needed. [44]

Incident: sportsbook acceptance issues (limits / voids) - This is outside the scanner's control, but your product should help you manage it: - Track per-operator "soft limit" notes. - Let you lower max stake for a book. - Tag books as "low reliability" to reduce alerting frequency.

Bankroll tracking - Maintain a manual balance per operator. - Persist per-operator max stake and reserve buffer. - Stake sizing respects balances and buffers.

Acceptance criteria (MVP)

You can call MVP "done" when all are true:

1. The system continuously ingests odds for at least one league and at least 4 Iowa-legal operators from the regulator list. [1]
2. Normalization produces canonical records with stable IDs and timestamps.
3. Arb engine detects and alerts:
4. 2-way moneyline arbs
5. 3-way 1X2 arbs where available
6. Telegram alerts send via documented Bot API patterns; message formatting is readable; retries handle 429 with backoff. [8]
7. Dedupe prevents spam.
8. Iowa restriction filters exist, including blocking Iowa college player prop wagers. [5]
9. Dashboard shows at least:
10. ingestion health
11. arbs found vs alerted vs filtered
12. last alert times
13. You can pause/resume alerts safely.

Ongoing KPIs you should track from day one

- p50/p95 time-to-alert (ingest → Telegram delivered)
- false positives by root cause (stale, mismatch, suspended, stake infeasible)
- vendor 429 rate and quota usage
- top leagues/markets by arb frequency
- dedupe suppression rate (to prove throttling is working)

If you want one more constraint baked into the PRD before you build: decide whether you want **live (in-play) arbing in MVP**. Many feeds publish slower polling intervals for in-play odds or require streaming tiers to be practical, so this single decision changes your vendor choice and performance targets materially. [58]

---

[1] [11] [14] [15] [24] https://irgc.iowa.gov/licensing-information/sports-wagering
https://irgc.iowa.gov/licensing-information/sports-wagering

[2] [5] https://irgc.iowa.gov/media/245/download?inline=
https://irgc.iowa.gov/media/245/download?inline=

[3] [20] [52] [58] https://the-odds-api.com/sports-odds-data/update-intervals.html
https://the-odds-api.com/sports-odds-data/update-intervals.html

[4] [7] [29] https://the-odds-api.com/guide/rate-limit.html
https://the-odds-api.com/guide/rate-limit.html

[6] [9] [13] [37] [47] [55] https://core.telegram.org/bots/faq
https://core.telegram.org/bots/faq

[8] [19] [54] [56] https://core.telegram.org/bots/api
https://core.telegram.org/bots/api

[10] [21] [26] [42] [48] [49] https://unabated.com/get-unabated-api
https://unabated.com/get-unabated-api

[12] [17] [38] [39] [41] https://sportsdata.io/live-odds-api
https://sportsdata.io/live-odds-api

[16] [31] [43] [44] https://docs.therundown.io/rate-limits
https://docs.therundown.io/rate-limits

[18] [28] [30] https://the-odds-api.com/
https://the-odds-api.com/

[22] [33] [35] https://developer.opticodds.com/docs/api-faq
https://developer.opticodds.com/docs/api-faq

[23] [27] https://the-odds-api.com/liveapi/guides/v4/
https://the-odds-api.com/liveapi/guides/v4/

[25] https://docs.sportradar.com/uof
https://docs.sportradar.com/uof

[32] [53] [57] https://developer.opticodds.com/docs/data-ingestion
https://developer.opticodds.com/docs/data-ingestion

[34] https://opticodds.com/sports-betting-api
https://opticodds.com/sports-betting-api

[36] https://opticodds.com/pricing
https://opticodds.com/pricing

[40] https://sportsdata.io/apis
https://sportsdata.io/apis

45 https://docs.therundown.io/
https://docs.therundown.io/

46 https://therundown.io/api
https://therundown.io/api

50 51 https://docs.unabated.com/
https://docs.unabated.com/